

Turing Function Calling Test Preparation Guide

Test Format Overview

You'll be given:

1. **User Query** - What the user wants to accomplish
2. **Available Tools** - List of functions you can call
3. **Task** - Identify the best function(s) and construct proper JSON payload(s)

Step-by-Step Approach

Step 1: Analyze the User Query

- Identify the **main intent** (what does the user want?)
- Break down **multiple requests** if present
- Note **key parameters** mentioned (locations, IDs, filters, etc.)

Step 2: Review Available Tools ⚠ CRITICAL

- **SCROLL AND READ ALL TOOLS** - The list may be long!
- Don't assume tools are missing - check the entire list first
- Read each function name carefully
- Match function purpose to user intent
- Check if multiple functions are needed
- **Common trap:** Answering N/A when the right tool is further down the list

Step 3: Construct JSON Payload

- Follow the exact schema provided
- Use correct data types (strings, numbers, booleans, arrays, objects)
- **Parameter names are typically camelCase** (userId, maxResults, includeSpamTrash)
- Include all required parameters
- Use proper nesting for objects
- **For multi-step questions:** First call may have payload, subsequent calls might be "None"

Step 4: Validate

- Check JSON syntax (commas, quotes, brackets)
- Verify parameter names match schema exactly
- Ensure no extra commentary or natural language

Common Patterns

Pattern 1: Single Function Call

Query: "What's the weather in Boston?" **Available:** `get_weather` **Answer:**

```
{
  "selected_function": "get_weather",
  "arguments": {
    "location": "Boston",
    "unit": "celsius"
  }
}
```

Pattern 2: Multiple Function Calls

Query: "Delete messages 101, 102, and 103, then send my draft" **Available:** `batch_delete_messages`, `send_draft` **Answer:**

```
[
  {
    "selected_function": "batch_delete_messages",
    "arguments": {
      "message_ids": [101, 102, 103]
    }
  },
  {
    "selected_function": "send_draft",
    "arguments": {
      "draft_id": null
    }
  }
]
```

Pattern 3: No Matching Function

Query: "Can you archive my old emails?" **Available:** `list_messages`, `send_draft`, `delete_message` **Answer:**

N/A

Pattern 4: Multiple Actions from Extended Tool List

Query: "Can you grab my profile info real quick and stop watching my mailbox? I don't need live updates right now." **Available:** `send_draft`, `list_messages`, `batch_delete_messages`, `get_vacation_settings`, `update_vacation_settings`, `get_user_profile`, `stop_user_watch` **Answer:**

```
[
  {
    "selected_function": "get_user_profile",
    "arguments": {}
  },
]
```

```
{
  "selected_function": "stop_user_watch",
  "arguments": {}
}
```

⚠ **Key Lesson:** Always scroll through the ENTIRE tool list! The tools you need might be at the bottom!

Practice Examples

Example 1: Email Management

Query: "Show me all my draft emails" **Available Tools:**

- list_drafts
- send_draft
- list_messages
- batch_delete_messages

Solution:

```
{
  "selected_function": "list_drafts",
  "arguments": {}
}
```

Explanation: User wants to see drafts, `list_drafts` is the exact match.

Example 2: Weather Query

Query: "What's the temperature in New York in Fahrenheit?" **Available Tools:**

- get_weather (parameters: location, unit)
- get_forecast (parameters: location, days)

Solution:

```
{
  "selected_function": "get_weather",
  "arguments": {
    "location": "New York",
    "unit": "fahrenheit"
  }
}
```

Explanation: User wants current weather (not forecast), specific location and unit provided.

Example 3: Multiple Actions

Query: "Delete messages 45 and 67, then list all my drafts" **Available Tools:**

- list_drafts
- batch_delete_messages
- list_messages

Solution:

```
[
  {
    "selected_function": "batch_delete_messages",
    "arguments": {
      "message_ids": [45, 67]
    }
  },
  {
    "selected_function": "list_drafts",
    "arguments": {}
  }
]
```

Explanation: Two distinct actions requested, execute in order mentioned.

Example 4: Implied Parameters

Query: "Send my latest draft" **Available Tools:**

- send_draft (parameters: draft_id [optional])
- list_drafts

Solution:

```
{
  "selected_function": "send_draft",
  "arguments": {
    "draft_id": null
  }
}
```

Explanation: No specific draft ID given, but function can handle latest draft with null/no parameter.

Example 5: No Match

Query: "Unsubscribe me from all newsletters" **Available Tools:**

- list_messages
- batch_delete_messages
- send_draft

Solution:

N/A

Explanation: No function handles subscriptions - available tools only handle messages/drafts.

Common Mistakes to Avoid

✗ Mistake 1: Adding Commentary

```
{
  "selected_function": "get_weather", // This will get the weather
  "arguments": {
    "location": "Boston" // User mentioned Boston
  }
}
```

Why wrong: JSON doesn't support comments in this format.

✗ Mistake 2: Wrong Data Types

```
{
  "selected_function": "batch_delete_messages",
  "arguments": {
    "message_ids": "45, 67" // Should be array, not string
  }
}
```

Should be: "message_ids": [45, 67]

✗ Mistake 3: Misspelled Parameters

```
{
  "selected_function": "get_weather",
  "arguments": {
    "loc": "Boston", // Should be "location"
    "temp_unit": "celsius" // Should be "unit"
  }
}
```

✗ Mistake 4: Forcing a Function Call

Query: "Can you remind me tomorrow?" **Available:** `send_email`, `delete_email` **Wrong:** Trying to use `send_email` creatively **Correct:** Answer `N/A` - no reminder function available

✗ Mistake 5: Missing Required Parameters

```
{
  "selected_function": "get_weather",
  "arguments": {} // Missing required "location"
}
```

Key Tips for Success

1. **Read Twice, Write Once** - Fully understand query and available tools before answering
 2. **SCROLL THROUGH ALL TOOLS** - Don't stop at the first few! The tool you need might be at the bottom
 3. **Match Intent, Not Keywords** - Focus on what user wants to accomplish
 4. **Respect the Schema** - Exact parameter names, correct types
 5. **Be Honest with N/A** - Don't force functions that don't match (but only after checking ALL tools!)
 6. **Order Matters** - For multiple calls, sequence should be logical
 7. **Valid JSON Only** - No comments, trailing commas, or syntax errors
 8. **Check Edge Cases** - null values, empty arrays, optional parameters
-

Practice Exercises

Exercise 1

Query: "List all messages in my inbox" **Available:** `list_messages`, `send_draft`, `list_drafts` **Your Answer:** _____

Exercise 2

Query: "Get the 5-day forecast for London in Celsius" **Available:** `get_weather` (location, unit), `get_forecast` (location, days, unit) **Your Answer:** _____

Exercise 3

Query: "Can you archive my emails and backup my contacts?" **Available:** `list_messages`, `batch_delete_messages` **Your Answer:** _____

Exercise 4

Query: "Delete draft 123 and draft 456" **Available:** `delete_draft` (draft_id), `batch_delete_drafts` (draft_ids) **Your Answer:** _____

Exercise 5 - EXTENDED TOOL LIST ⚠

Query: "Show me all my email labels and create a new filter" **Available:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads **Your Answer:** _____

Exercise 6 - EXTENDED TOOL LIST ⚠

Query: "I need to set up email forwarding to my work address and check my current IMAP configuration"
Available: send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads **Your Answer:** _____

Answer Key

Exercise 1 Answer:

```
{
  "selected_function": "list_messages",
  "arguments": {}
}
```

Exercise 2 Answer:

```
{
  "selected_function": "get_forecast",
  "arguments": {
    "location": "London",
    "days": 5,
    "unit": "celsius"
  }
}
```

Exercise 3 Answer:

N/A

(No archive or backup functions available)

Exercise 4 Answer:

```
{
  "selected_function": "batch_delete_drafts",
  "arguments": {
```

```
"draft_ids": [123, 456]
}
```

(Prefer batch operation when available)

Exercise 5 Answer:

```
[
  {
    "selected_function": "list_labels",
    "arguments": {}
  },
  {
    "selected_function": "create_filter",
    "arguments": {}
  }
]
```

(Two separate actions - list labels AND create filter. Both tools were available in the extended list!)

Exercise 6 Answer:

```
[
  {
    "selected_function": "create_forwarding_address",
    "arguments": {}
  },
  {
    "selected_function": "get_imap_settings",
    "arguments": {}
  }
]
```

(User wants to SET UP forwarding and CHECK/GET IMAP settings - both available!)

Final Checklist Before Submitting

- ☐ User query fully understood
- ☐ **ALL available tools reviewed (scrolled to the bottom!)**
- ☐ Best matching function(s) identified
- ☐ All required parameters included
- ☐ Correct data types used
- ☐ Valid JSON syntax (use JSON validator if available)
- ☐ No extra commentary or natural language
- ☐ Multiple actions in correct sequence

- ☐ If no match, answered "N/A" (only after checking ALL tools)

Most Common Test Trap

THE INCOMPLETE TOOL LIST TRAP

Many test-takers fail because they:

1. See the first 3-4 tools
2. Don't see a match
3. Answer "N/A"
4. **FAIL** - The right tool was tool #7!

Always scroll down and review the complete tool list before deciding!

Good luck with your Turing test preparation!

Advanced Practice Scenarios

Scenario 1: Email Organization

Query: "Remove the 'Promotions' label from my account" **Available Tools:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

Solution:

```
{
  "selected_function": "delete_label",
  "arguments": {
    "label_name": "Promotions"
  }
}
```

Note: Assumed parameter based on function name. In real test, schema would be provided.

Scenario 2: Thread Management

Query: "Show me all conversation threads in my mailbox" **Available Tools:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

Solution:

```
{
  "selected_function": "list_threads",
  "arguments": {}
}
```

Why not list_messages? User specifically asked for "threads" (conversations), not individual messages.

`list_threads` is the better match.

Scenario 3: Vacation Setup

Query: "I'm going on vacation - turn on my out-of-office reply" **Available Tools:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter

Solution:

```
{
  "selected_function": "update_vacation_settings",
  "arguments": {
    "enabled": true
  }
}
```

Key insight: User wants to TURN ON vacation mode = UPDATE settings, not GET settings.

Scenario 4: Multiple Configuration Changes

Query: "Enable IMAP access and set up auto-forwarding to john@company.com" **Available Tools:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

Solution:

```
[
  {
    "selected_function": "update_imap_settings",
    "arguments": {
      "enabled": true
    }
  },
  {
    "selected_function": "create_forwarding_address",
    "arguments": {
      "forwarding_email": "john@company.com"
    }
  }
]
```

```
}
]
```

Two actions: Enable IMAP (update) + Create forwarding (to specific address)

Scenario 5: Tricky Wording

Query: "What's my current forwarding setup?" **Available Tools:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

Solution:

```
{
  "selected_function": "get_forwarding_address",
  "arguments": {}
}
```

Key distinction: "What's my current..." = GET (read), not CREATE or UPDATE. Don't confuse checking settings with changing them!

Scenario 6: Bulk Operations

Query: "Delete messages 100, 200, 300, 400, and 500" **Available Tools:** send_draft, list_messages, batch_delete_messages, delete_message, get_vacation_settings

Solution:

```
{
  "selected_function": "batch_delete_messages",
  "arguments": {
    "message_ids": [100, 200, 300, 400, 500]
  }
}
```

Why batch? Multiple items to delete. If both `delete_message` and `batch_delete_messages` exist, prefer batch for efficiency.

Pattern Recognition Guide

GET vs UPDATE vs CREATE

User Says	Intent	Function Pattern
-----------	--------	------------------

User Says	Intent	Function Pattern
"What's my...", "Show me...", "Check..."	READ	<code>get_*</code> or <code>list_*</code>
"Change...", "Update...", "Enable...", "Disable..."	MODIFY	<code>update_*</code>
"Set up...", "Add...", "Create..."	CREATE	<code>create_*</code>
"Remove...", "Delete..."	DELETE	<code>delete_*</code>
"Turn on...", "Turn off..."	MODIFY	<code>update_*</code> with boolean

Singular vs Batch Operations

Scenario	Prefer
Multiple items mentioned	<code>batch_*</code> function
Single item	Regular function
Both available for multiple items	<code>batch_*</code>

Similar Function Names - How to Choose

When you see similar functions like:

- `get_vacation_settings` vs `update_vacation_settings`
- `create_filter` vs `delete_filter`
- `list_labels` vs `delete_label`

Ask yourself:

1. Is the user reading/checking (GET/LIST)?
2. Is the user changing/enabling (UPDATE)?
3. Is the user adding new (CREATE)?
4. Is the user removing (DELETE)?

Common Test Gotchas

Gotcha #1: Synonyms

Query: "Set up my out-of-office message" **Available:** `update_vacation_settings` **Trap:** Looking for `set_out_of_office` - doesn't exist! **Solution:** "Out-of-office" = vacation settings

Gotcha #2: Implicit Actions

Query: "I'm going on vacation next week" **Available:** `update_vacation_settings` **Trap:** No explicit request for action **Solution:** Context implies they want vacation mode enabled

Gotcha #3: Multiple Phrasings

Query: "Stop monitoring my inbox" **Available:** `stop_user_watch` **Trap:** Looking for `stop_inbox_monitoring` **Solution:** "monitoring" = "watching" in this context

Gotcha #4: Lists vs Items

Query: "Show me my email thread for order #12345" **Available:** `list_threads`, `get_thread` **Trap:** User said "thread" (singular) but you pick `list_threads` (plural) **Solution:** Singular request = singular function if available

Gotcha #5: Assuming Parameters

Query: "Create a filter" **Available:** `create_filter` **Trap:** Adding parameters not mentioned (filter criteria, etc.) **Solution:** If schema allows empty or has defaults, use `{}`

Quick Reference: Email API Functions

Based on the extended tool list you've shown:

Function	Purpose	Typical Arguments
<code>send_draft</code>	Send a draft email	<code>draft_id</code>
<code>list_messages</code>	List emails	<code>filters</code> , <code>limit</code>
<code>batch_delete_messages</code>	Delete multiple emails	<code>message_ids</code> (array)
<code>get_vacation_settings</code>	Check auto-reply status	<code>none</code>
<code>update_vacation_settings</code>	Enable/disable auto-reply	<code>enabled</code> , <code>message</code>
<code>get_user_profile</code>	Get user account info	<code>none</code>
<code>stop_user_watch</code>	Stop inbox monitoring	<code>none</code>
<code>list_labels</code>	Show all labels/folders	<code>none</code>
<code>delete_label</code>	Remove a label	<code>label_name</code> or <code>label_id</code>
<code>create_filter</code>	Add email filter rule	<code>criteria</code> , <code>actions</code>
<code>create_forwarding_address</code>	Set up forwarding	<code>forwarding_email</code>
<code>get_forwarding_address</code>	Check forwarding setup	<code>none</code>
<code>get_imap_settings</code>	Check IMAP config	<code>none</code>
<code>update_imap_settings</code>	Change IMAP config	<code>enabled</code> , other settings
<code>list_threads</code>	Show conversation threads	<code>filters</code> , <code>limit</code>

Test-Taking Strategy

Time Management

1. **First Pass (30 seconds):** Read query, scroll through ALL tools
2. **Second Pass (30 seconds):** Match intent to function(s)
3. **Third Pass (60 seconds):** Construct JSON payload
4. **Final Check (30 seconds):** Validate syntax and parameters

Decision Tree

```
1. Read query → What does user want?
  ↓
2. Scroll through ALL tools → Which function(s) match?
  ↓
3. Do any tools match?
  ↓
  YES → Continue
  NO → Answer N/A
  ↓
4. Single action or multiple?
  ↓
  Single → One JSON object
  Multiple → Array of JSON objects
  ↓
5. What parameters are mentioned?
  ↓
6. Build JSON with exact parameter names
  ↓
7. Validate JSON syntax
  ↓
8. Submit
```

Final Pro Tips

1. **Read the schema carefully** - Parameter names must match exactly
2. **Arrays need brackets** - `[100, 200]` not `"100, 200"`
3. **Strings need quotes** - `"Boston"` not `Boston`
4. **Booleans don't need quotes** - `true` not `"true"`
5. **null is a valid value** - When optional parameters aren't specified
6. **Order matters** - Execute actions in logical sequence
7. **One task = one function** - Don't try to make one function do multiple things
8. **Multiple tasks = multiple functions** - Use array format
9. **No such thing as "close enough"** - Function names must match exactly
10. **When in doubt, scroll again** - The tool you need might be there!

Mock Test Questions

Try these on your own before checking answers:

Mock Question 1

Query: "Turn off my vacation responder and show me all my email threads" **Available:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

► Click to see answer

```
[
  {
    "selected_function": "update_vacation_settings",
    "arguments": {
      "enabled": false
    }
  },
  {
    "selected_function": "list_threads",
    "arguments": {}
  }
]
```

Mock Question 2

Query: "What labels do I have set up in my account?" **Available:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter

► Click to see answer

```
{
  "selected_function": "list_labels",
  "arguments": {}
}
```

Mock Question 3

Query: "Archive all my promotional emails" **Available:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter

► Click to see answer

N/A

(No archive function available)

Mock Question 4

Query: "I need to check my IMAP settings and create a forwarding rule to backup@example.com" **Available:** send_draft, list_messages, batch_delete_messages, get_vacation_settings, update_vacation_settings, get_user_profile, stop_user_watch, list_labels, delete_label, create_filter, create_forwarding_address, get_forwarding_address, get_imap_settings, update_imap_settings, list_threads

► Click to see answer

```
[
  {
    "selected_function": "get_imap_settings",
    "arguments": {}
  },
  {
    "selected_function": "create_forwarding_address",
    "arguments": {
      "forwarding_email": "backup@example.com"
    }
  }
]
```

You're now ready for your Turing test! Remember: **Scroll through ALL tools first!** 🔄

EXTENDED FUNCTION LIST - Security & Advanced Features

Additional Tools Discovered:

Based on extended scrolling, here are MORE functions you might encounter:

Function	Purpose	Category
email_create_cse_keypair	Create client-side encryption keys	Security
email_enable_cse_keypair	Enable encryption keypair	Security
email_get_cse_keypair	Retrieve encryption keypair info	Security
create_send_as_alias	Create alias for sending emails	Identity
insert_smime_config	Add S/MIME encryption config	Security
set_default_smime_config	Set default S/MIME settings	Security
verify_send_as_alias	Verify send-as alias ownership	Identity

Key Takeaway: Real tests may have 20+ functions. ALWAYS scroll to the very bottom!

Advanced Security Scenarios

Scenario 7: Encryption Setup

Query: "Set up client-side encryption for my emails" **Available Tools:** [previous tools] + email_create_cse_keypair, email_enable_cse_keypair, email_get_cse_keypair, create_send_as_alias, insert_smime_config, set_default_smime_config, verify_send_as_alias

Solution:

```
[
  {
    "selected_function": "email_create_cse_keypair",
    "arguments": {}
  },
  {
    "selected_function": "email_enable_cse_keypair",
    "arguments": {}
  }
]
```

Reasoning: "Set up" encryption requires CREATE then ENABLE - two sequential actions.

Scenario 8: Alias Management

Query: "Add support@company.com as a sending address and verify it" **Available Tools:** [previous tools] + email_create_cse_keypair, email_enable_cse_keypair, email_get_cse_keypair, create_send_as_alias, insert_smime_config, set_default_smime_config, verify_send_as_alias

Solution:

```
[
  {
    "selected_function": "create_send_as_alias",
    "arguments": {
      "alias_email": "support@company.com"
    }
  },
  {
    "selected_function": "verify_send_as_alias",
    "arguments": {
      "alias_email": "support@company.com"
    }
  }
]
```

Reasoning: User wants to ADD (create) and VERIFY - two actions, same parameter used twice.

Scenario 9: Security Configuration

Query: "What encryption keys do I currently have set up?" **Available Tools:** [previous tools] + email_create_cse_keypair, email_enable_cse_keypair, email_get_cse_keypair, create_send_as_alias, insert_smime_config, set_default_smime_config, verify_send_as_alias

Solution:

```
{
  "selected_function": "email_get_cse_keypair",
  "arguments": {}
}
```

Reasoning: "What...do I have" = GET/READ operation, not CREATE or ENABLE.

Scenario 10: S/MIME Setup

Query: "Configure S/MIME encryption as my default security method" **Available Tools:** [previous tools] + email_create_cse_keypair, email_enable_cse_keypair, email_get_cse_keypair, create_send_as_alias, insert_smime_config, set_default_smime_config, verify_send_as_alias

Solution:

```
[
  {
    "selected_function": "insert_smime_config",
    "arguments": {}
  },
  {
    "selected_function": "set_default_smime_config",
    "arguments": {}
  }
]
```

Reasoning: Configure = INSERT/ADD the config, then SET it as DEFAULT - two steps.

Acronyms & Technical Terms You Should Know

When you see these in queries or function names:

Term	Meaning	Common Usage
CSE	Client-Side Encryption	Encrypting emails on user's device
S/MIME	Secure/Multipurpose Internet Mail Extensions	Email encryption standard
IMAP	Internet Message Access Protocol	Email retrieval protocol

Term	Meaning	Common Usage
SMTP	Simple Mail Transfer Protocol	Email sending protocol
Alias	Alternative email address	Sending from different address
Keypair	Public + Private encryption keys	Used for encryption/decryption
Thread	Email conversation	Multiple related messages
Draft	Unsent email	Email saved but not sent
Label	Email folder/category	Like Gmail labels
Filter	Automatic email rule	Auto-organize emails

Query Translation Examples:

User Says	They Mean	Look For
"Set up encryption"	Create & enable keypair	<code>create_cse_keypair</code> + <code>enable_cse_keypair</code>
"Send as my work email"	Use alias	<code>create_send_as_alias</code>
"Check my security setup"	Get encryption info	<code>get_cse_keypair</code> or <code>get_smime_config</code>
"Configure secure email"	Set up S/MIME	<code>insert_smime_config</code>
"Use encrypted email by default"	Set default encryption	<code>set_default_*_config</code>

More Complex Multi-Step Examples

Example A: Complete Encryption Setup

Query: "I need full email encryption - create keys, enable them, and make it my default setup"

Solution:

```
[
  {
    "selected_function": "email_create_cse_keypair",
    "arguments": {}
  },
  {
    "selected_function": "email_enable_cse_keypair",
    "arguments": {}
  },
  {
    "selected_function": "insert_smime_config",
    "arguments": {}
  },
  {
```

```
    "selected_function": "set_default_smime_config",
    "arguments": {}
  }
]
```

Breaking it down: 4 separate actions in logical order - CREATE keys → ENABLE keys → INSERT S/MIME → SET as DEFAULT

Example B: Alias Verification Flow

Query: "Can you verify my sales@company.com alias is working?"

Solution:

```
{
  "selected_function": "verify_send_as_alias",
  "arguments": {
    "alias_email": "sales@company.com"
  }
}
```

Note: User wants to VERIFY (not create), and provides specific email address as parameter.

Example C: Security Audit

Query: "Show me my current encryption keypair and S/MIME configuration"

Solution:

```
[
  {
    "selected_function": "email_get_cse_keypair",
    "arguments": {}
  },
  {
    "selected_function": "get_smime_config",
    "arguments": {}
  }
]
```

Note: Two separate GET operations - both reading configuration, no modifications.

Function Naming Patterns - Advanced

Pattern: Domain Prefix

Some functions have prefixes indicating their category:

- `email_*` → Email-specific operations (often security)
 - `email_create_cse_keypair`
 - `email_enable_cse_keypair`
 - `email_get_cse_keypair`
- No prefix → General mail operations
 - `send_draft`
 - `list_messages`
 - `create_filter`

When matching: If query mentions "email encryption" or "email security", look for `email_*` functions first.

Pattern: Action_Object

Most functions follow: `action_object` format

- `create_send_as_alias` = CREATE a send-as ALIAS
- `verify_send_as_alias` = VERIFY a send-as ALIAS
- `insert_smime_config` = INSERT smime CONFIG
- `set_default_smime_config` = SET DEFAULT smime CONFIG

Strategy: Identify the ACTION (verb) and OBJECT (noun) in the user query, then look for `action_object` pattern.

Mock Test - Extended Security Scenarios

Mock Question 5

Query: "I want to start using client-side encryption - set it up for me" **Available:** [all 22+ functions including security tools]

► Click to see answer

```
[
  {
    "selected_function": "email_create_cse_keypair",
    "arguments": {}
  },
  {
    "selected_function": "email_enable_cse_keypair",
    "arguments": {}
  }
]
```

Explanation: "Set up" encryption = CREATE then ENABLE. CSE = Client-Side Encryption.

Mock Question 6

Query: "Add info@startup.com as one of my send-as addresses" **Available:** [all 22+ functions including alias tools]

► Click to see answer

```
{
  "selected_function": "create_send_as_alias",
  "arguments": {
    "alias_email": "info@startup.com"
  }
}
```

Explanation: "Add as send-as address" = CREATE alias. Specific email provided as parameter.

Mock Question 7

Query: "Check if my team@company.com sending alias has been verified yet" **Available:** [all 22+ functions including alias tools]

► Click to see answer

```
{
  "selected_function": "verify_send_as_alias",
  "arguments": {
    "alias_email": "team@company.com"
  }
}
```

Explanation: "Check if verified" = VERIFY operation (not GET or CREATE).

Mock Question 8

Query: "Turn on S/MIME and make it my default encryption method" **Available:** [all 22+ functions including S/MIME tools]

► Click to see answer

```
[
  {
    "selected_function": "insert_smime_config",
    "arguments": {}
  },
  {
    "selected_function": "set_default_smime_config",
```

```
    "arguments": {}  
  }  
]
```

Explanation: Two actions - INSERT (turn on) S/MIME config, then SET as DEFAULT.

Updated Complete Function Reference

Full Tool List (22+ Functions)

Messaging (4)

- send_draft
- list_messages
- batch_delete_messages
- list_threads

Account & Monitoring (4)

- get_user_profile
- stop_user_watch
- get_vacation_settings
- update_vacation_settings

Organization (3)

- list_labels
- delete_label
- create_filter

Forwarding (2)

- create_forwarding_address
- get_forwarding_address

Mail Protocol Settings (2)

- get_imap_settings
- update_imap_settings

Client-Side Encryption (3)

- email_create_cse_keypair
- email_enable_cse_keypair
- email_get_cse_keypair

Send-As Aliases (2)

- create_send_as_alias
- verify_send_as_alias

S/MIME Encryption (2)

- insert_smime_config
 - set_default_smime_config
-

Critical Success Factors - Updated

☑ DO:

1. Scroll through **every single function** before deciding
2. Match user intent (action + object) to function name
3. Use arrays [] for multiple function calls
4. Include exact parameter names from schema
5. Understand domain terminology (CSE, S/MIME, IMAP, etc.)
6. Break down complex requests into multiple steps
7. Sequence operations logically (create → enable → set default)

✗ DON'T:

1. Stop scrolling after first 5-10 functions
 2. Answer N/A without checking the complete list
 3. Guess parameter names or data types
 4. Combine multiple actions into one function call
 5. Add parameters not mentioned in the query
 6. Use wrong action verb (create vs get vs update)
 7. Ignore technical acronyms (they're clues!)
-

Final Exam Readiness Checklist

- ☐ I understand GET vs CREATE vs UPDATE vs DELETE patterns
- ☐ I know when to use arrays for multiple function calls
- ☐ I can identify technical acronyms (CSE, S/MIME, IMAP)
- ☐ I always scroll through the complete tool list
- ☐ I check parameter types (string, number, boolean, array)
- ☐ I sequence multi-step operations logically
- ☐ I know when to answer N/A (only after checking ALL tools)
- ☐ I can match user intent to function names
- ☐ I validate JSON syntax before submitting
- ☐ I understand the difference between similar functions (create vs verify)

If you checked all boxes, you're ready for the Turing test! 🚀

Real Test Format Examples

Example Response Format (From Actual Test)

When tests require multiple tool calls, you'll format them like this:

Tool Call 1:

```
list_drafts
```

Tool Call 1 Payload:

```
{
  "userId": "alice@example.com",
  "maxResults": 100,
  "includeSpamTrash": false
}
```

Tool Call 2:

```
Select an option (or "None" if no second call needed)
```

Parameter Naming Conventions

Based on actual test examples, parameters typically follow these patterns:

CamelCase (Most Common)

```
{
  "userId": "value",
  "maxResults": 100,
  "includeSpamTrash": false,
  "labelId": "INBOX",
  "messageIds": [1, 2, 3]
}
```

Common Parameter Names You'll See:

Parameter	Type	Example	Common In
userId	string	"user@example.com"	Most operations
maxResults	number	100	List operations
includeSpamTrash	boolean	false	Message lists
messageIds	array	[123, 456]	Batch operations
labelId	string	"INBOX"	Label operations

Parameter	Type	Example	Common In
labelName	string	"Work"	Label operations
draftId	string/number	"draft_123"	Draft operations
enabled	boolean	true	Settings operations
forwardingEmail	string	"backup@example.com"	Forwarding
aliasEmail	string	"team@company.com"	Alias operations

Data Type Rules (CRITICAL!)

Strings - Always Quoted

```
{
  "userId": "alice@example.com",
  "userId": alice@example.com,
  "labelName": "Work Projects"
}
```

☒ Correct

☒ Correct (spaces allowed in strings)

☒ Wrong - missing quotes

Numbers - Never Quoted

```
{
  "maxResults": 100,
  "maxResults": "100",
  "limit": 50
}
```

☒ Correct

☒ Correct

☒ Wrong - should be number, not string

Booleans - Never Quoted

```
{
  "enabled": true,
  "enabled": "true",
  "includeSpamTrash": false
}
```

☒ Correct

☒ Correct

☒ Wrong - should be boolean, not string

Arrays - Use Square Brackets

```
{
  "messageIds": [123, 456, 789],
  "messageIds": "123, 456, 789",
  "labels": ["Work", "Important"],
}
```

☒ Correct - array of numbers

☒ Correct - array of strings

☒ Wrong - string, not array

```
"labels": "Work, Important"
}
```

✗ Wrong - string, not array

null - When Optional Parameter Not Provided

```
{
  "draftId": null,
  "draftId": "null",
  "draftId":
}
```

✓ Correct - optional parameter
✗ Wrong - this is string "null", not null value
✗ Wrong - incomplete

Multi-Step Response Format

When You Need Multiple Tool Calls

Scenario: "List my drafts for alice@example.com and then send the first one"

Your Response:

Tool Call 1:

```
list_drafts
```

Tool Call 1 Payload:

```
{
  "userId": "alice@example.com",
  "maxResults": 1
}
```

Tool Call 2:

```
send_draft
```

Tool Call 2 Payload:

```
{
  "draftId": null
}
```

(null = use first/latest draft)

When You Only Need One Tool Call

Scenario: "Show me my vacation settings"

Your Response:

Tool Call 1:

```
get_vacation_settings
```

Tool Call 1 Payload:

```
{}
```

(Empty object if no parameters needed)

Tool Call 2:

```
None
```

(Or select "None" from dropdown)

Complete Real-World Example

Full Test Question Format

User Query: "Can you list all drafts for john@company.com, showing up to 50 results, and don't include spam?"

Available Tools:

- list_drafts
- send_draft
- list_messages
- batch_delete_messages
- get_vacation_settings

Tool Schema for list_drafts:

```
{
  "userId": "string (required)",
  "maxResults": "number (optional, default 100)",
```

```
"includeSpamTrash": "boolean (optional, default true)"
}
```

YOUR ANSWER:**Tool Call 1:**

```
list_drafts
```

Tool Call 1 Payload:

```
{
  "userId": "john@company.com",
  "maxResults": 50,
  "includeSpamTrash": false
}
```

Tool Call 2:

```
None
```

Explanation of Each Parameter:

- **userId**: String - exact email from query
- **maxResults**: Number - "up to 50 results"
- **includeSpamTrash**: Boolean - "don't include spam" = false

Common Payload Mistakes

✗ Mistake #1: Wrong Data Type

```
{
  "maxResults": "50",           // Wrong - should be number
  "includeSpamTrash": "false"  // Wrong - should be boolean
}
```

☑ Correct:

```
{
  "maxResults": 50,
  "includeSpamTrash": false
}
```

✗ Mistake #2: Inconsistent Casing

```
{
  "user_id": "alice@example.com",    // Wrong - should be camelCase
  "max_results": 100                // Wrong - should be camelCase
}
```

☑ Correct:

```
{
  "userId": "alice@example.com",
  "maxResults": 100
}
```

✗ Mistake #3: Array as String

```
{
  "messageIds": "123, 456, 789"    // Wrong - should be array
}
```

☑ Correct:

```
{
  "messageIds": [123, 456, 789]
}
```

✗ Mistake #4: Missing Required Parameters

```
{
  "maxResults": 50    // Missing required "userId"
}
```

☑ Correct:

```
{
  "userId": "user@example.com",

```

```
"maxResults": 50
}
```

✗ Mistake #5: Extra Comments

```
{
  "userId": "alice@example.com",    // User's email
  "maxResults": 100                // Limit results
}
```

☑ Correct:

```
{
  "userId": "alice@example.com",
  "maxResults": 100
}
```

Mock Test with Payload Requirements

Mock Question 9

User Query: "Get all messages for bob@workplace.com, limit to 25, and include spam folder"

Available Tools: list_messages (userId, maxResults, includeSpamTrash)

Tool Schema:

- **userId:** string (required)
- **maxResults:** number (optional, default 100)
- **includeSpamTrash:** boolean (optional, default false)

► Click to see answer

Tool Call 1:

```
list_messages
```

Tool Call 1 Payload:

```
{
  "userId": "bob@workplace.com",
  "maxResults": 25,
}
```

```
"includeSpamTrash": true
}
```

Tool Call 2:

None

Explanation:

- **userId**: "bob@workplace.com" (string, from query)
 - **maxResults**: 25 (number, not "25")
 - **includeSpamTrash**: true (boolean, "include spam" = true)
-

Mock Question 10

User Query: "Delete messages 501, 502, and 503 for user@domain.com"

Available Tools: batch_delete_messages (userId, messageIds)

Tool Schema:

- **userId**: string (required)
- **messageIds**: array of numbers (required)

► Click to see answer

Tool Call 1:

batch_delete_messages

Tool Call 1 Payload:

```
{
  "userId": "user@domain.com",
  "messageIds": [501, 502, 503]
}
```

Tool Call 2:

None

Explanation:

- `userId`: "user@domain.com" (string)
 - `messageIds`: [501, 502, 503] (array of numbers, not strings)
-

Payload Construction Checklist

Before submitting your payload, verify:

- ☐ All required parameters included
 - ☐ Parameter names match schema exactly (check camelCase)
 - ☐ Strings are quoted: `"value"`
 - ☐ Numbers are NOT quoted: `100`
 - ☐ Booleans are NOT quoted: `true` or `false`
 - ☐ Arrays use square brackets: `[1, 2, 3]`
 - ☐ Objects use curly braces: `{"key": "value"}`
 - ☐ No trailing commas after last parameter
 - ☐ No comments or extra text
 - ☐ Valid JSON syntax (use validator if available)
 - ☐ Selected "None" for Tool Call 2 if only one action needed
-

Test Interface Tips

Based on the screenshot, the test interface likely has:

1. **Dropdown menus** for selecting functions
2. **Text boxes** for entering JSON payloads
3. **"None" option** for subsequent tool calls when not needed
4. **Multiple tool call sections** (Tool Call 1, Tool Call 2, possibly more)

Strategy:

1. Select function from dropdown
 2. Type payload in text box
 3. If second tool needed, select from dropdown; otherwise select "None"
 4. Double-check JSON syntax before submitting
 5. Test in JSON validator if allowed (many online tools available)
-

JSON Syntax Quick Reference

Valid JSON Structure:

```
{
  "key1": "string value",
  "key2": 123,
  "key3": true,
  "key4": false,
  "key5": null,
  "key6": [1, 2, 3],
```

```
"key7": ["a", "b", "c"],
"key8": {
  "nested": "object"
}
```

Common Syntax Errors:

```
{
  "key1": "value",    // ✗ No comments allowed
  "key2": 'value',    // ✗ Use double quotes, not single
  "key3": value,      // ✗ Strings must be quoted
  "key4": true,       // ✗ No trailing comma on last item
}
```

You're now equipped with the complete payload formatting knowledge! 🎯